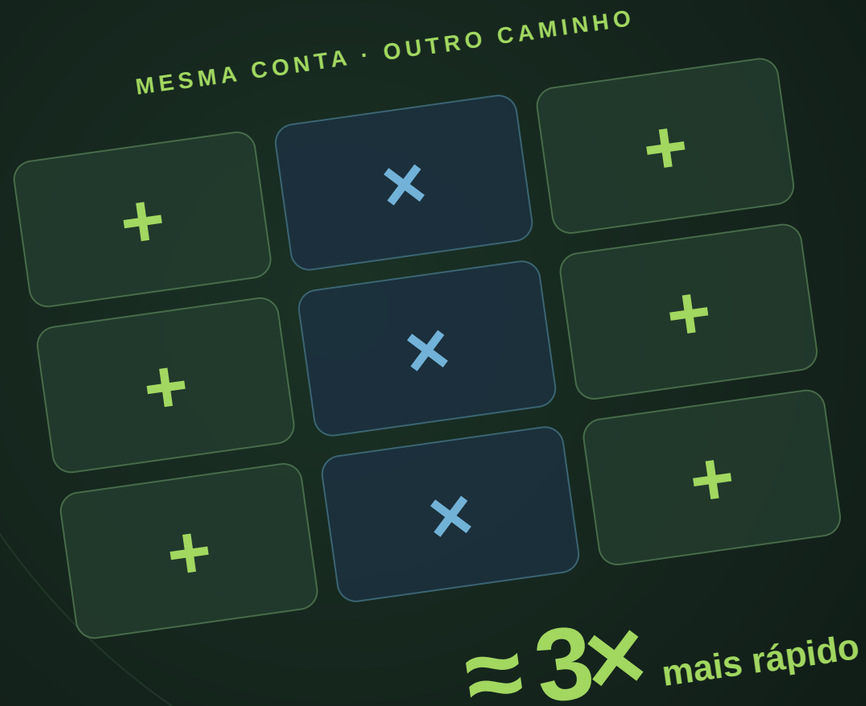
KERNELS / HPC (1/2)

Kernels CUDA na prática.

O que é um kernel — e por que mover dados pode custar mais que calcular.



Kernels CUDA na prática.

Como fazer **exatamente a mesma conta** muito mais rápido?

$$\vec{y} = (\vec{x} + \alpha)\beta + \gamma$$

x, y: vetores · α, β, γ: escalares

- Mesma matemática → Outro caminho pelos dados

0 FLOPs economizados → **~3×** mais rápido. **Como?**

FLOP conta. FLOP/s mede ritmo.

FLOP

Floating-Point Operation

Uma operação de ponto flutuante.

$a + b$ $a \times b$

1 FLOP cada

FLOP/s

ou FLOPS

O ritmo em que o hardware faz essas contas.

1 TFLOP/s $= 10^{12}\ \mathrm{FLOPs/s}$

x, y: VETORES · α, β, γ: ESCALARES

$$\vec{y} = (\vec{x} + \alpha)\beta + \gamma$$

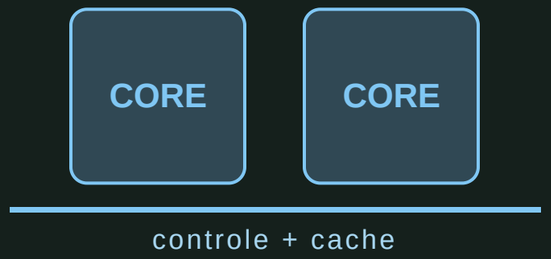
→ **3 FLOPs**

para cada elemento do vetor x

Contar FLOPs não diz, por si só, quanto tempo a operação leva.

Dois jeitos de gastar silício.

CPU **Baixa latência**



E se cada etapa depender da anterior?
A CPU prioriza resposta rápida por etapa.

GPU **Alto throughput**



E se houver muitas operações independentes?
A GPU usa muitas threads em paralelo.

CPU: otimizada para fazer uma sequência terminar rapidamente. GPU: otimiza muitas threads.

Onde minha thread trabalha?

GRID 2 blocks × 4 threads

BLOCK 0

0	1	2	3
t0	t1	t2	t3

BLOCK 1

4	5	6	7
t0	t1	t2	t3

números grandes = índice global · t = índice dentro do block

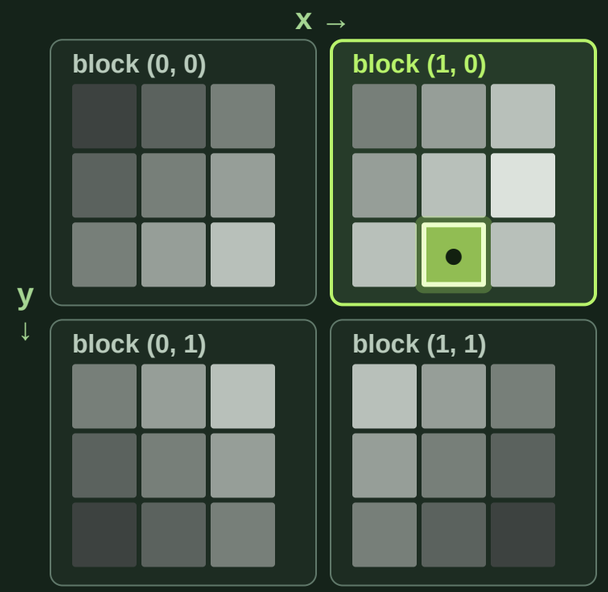
ÍNDICE GLOBAL $$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$$

O mesmo índice, agora em x e y.

Uma thread pode processar um pixel da imagem.

IMAGEM 6 × 6

4 blocos · 9 threads por bloco (3 × 3)



COORDENADAS GLOBAIS DO PIXEL

```
int x =
  blockIdx.x * blockDim.x + threadIdx.x;

int y =
  blockIdx.y * blockDim.y + threadIdx.y;
```

`block (1, 0)` `thread (1, 2)` → **pixel (4, 2)**

$$x = 1 \times 3 + 1 = 4 \qquad y = 0 \times 3 + 2 = 2$$

Grid e block podem ter mais de uma dimensão. Cada thread encontra seu (x, y).

CUDA também oferece o eixo z para dados 3D.

Um código. Muitas threads.

Kernel CUDA: função executada na GPU por muitas threads em paralelo.

HOST

CPU

lança

→

GPU

kernel

Single Program, Multiple Data (SPMD)

→

GRID

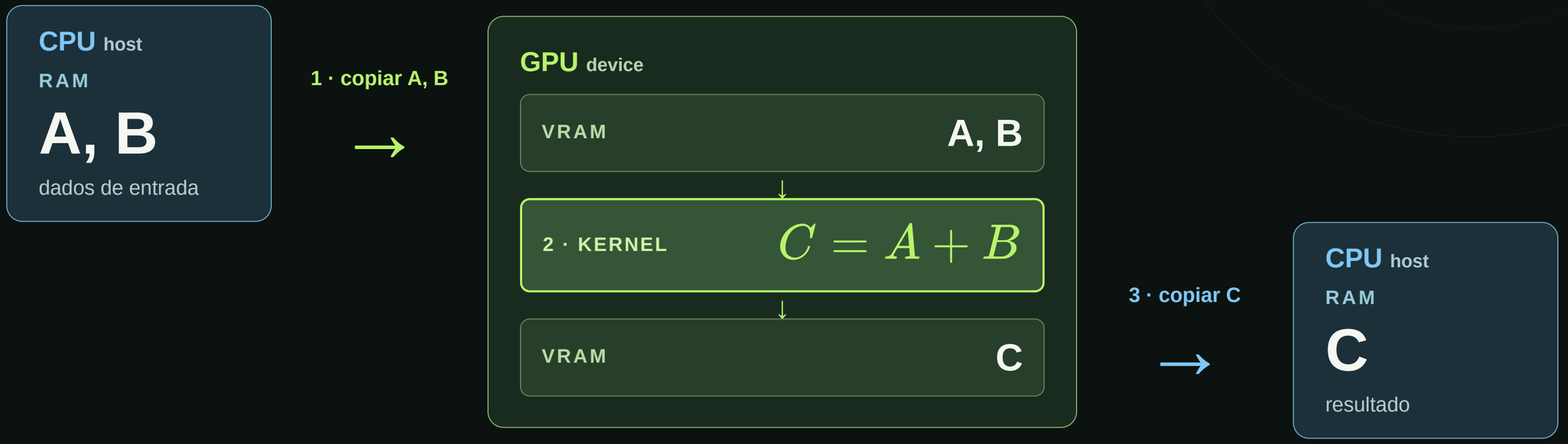
threads

dados diferentes

vector_add · soma vetorial

```
__global__ void vector_add(...) {
  int i = blockIdx.x * blockDim.x
        + threadIdx.x;
  if (i < N) C[i] = A[i] + B[i];
}
```

Uma soma. Dados vão e voltam.

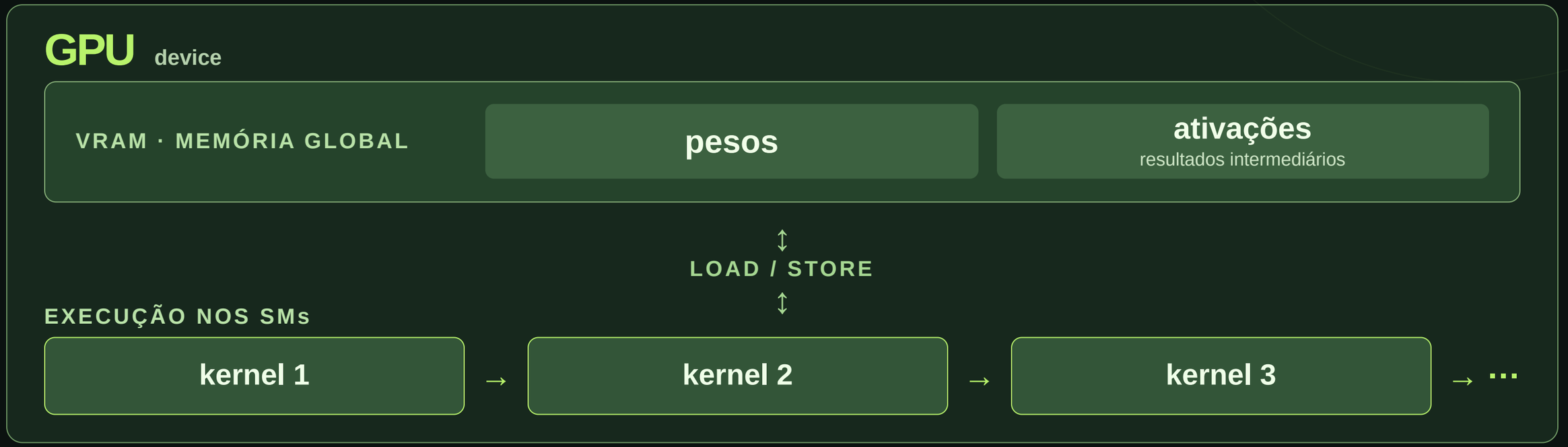


Pouco cálculo, muitos dados: alocação e cópias podem dominar o tempo total.

Medir só o kernel deixa esses custos de fora.

LLMs: dados ficam na GPU.

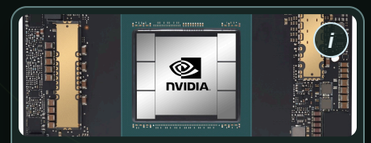
Pesos e ativações são tensores usados por muitos kernels para gerar tokens.



Na soma anterior, o resultado volta à CPU.
Na inferência, pesos e ativações permanecem na GPU entre os kernels.

Por que a GPU executa tantas threads?

A organização do hardware favorece tipos diferentes de trabalho.



NVIDIA A100 · Ampere

CPU

Poucos núcleos, mais recursos por núcleo

controle | ULAs (cálculos) | cache L1 | cache L2

controle | ULAs (cálculos) | cache L1 | cache L2

controle | ULAs (cálculos) | cache L1 | cache L2

controle | ULAs (cálculos) | cache L1 | cache L2

cache L3 compartilhado

memória RAM

GPU

Muitos SMs com unidades de cálculo

SM: warp | ULAs (cálculos) | controle | cache L1 | memória compart.

SM: ULAs (cálculos) | controle | cache L1 | memória compart.

SM: ULAs (cálculos) | controle | cache L1 | memória compart.

SM: ULAs (cálculos) | controle | cache L1 | memória compart.

SM: ULAs (cálculos) | controle | cache L1 | memória compart.

SM: ULAs (cálculos) | controle | cache L1 | memória compart.

cache L2 compartilhado

memória global / VRAM

Cada SM contém vários **núcleos CUDA**: unidades de processamento indicadas pela grade azul da GPU.

1.024 somas, 1.024 threads.

A CPU lança o kernel; CUDA organiza as threads em uma grid.



A CPU lança 4 blocks de 256 threads para somar os vetores.

A e B já estão na memória da GPU, conforme o fluxo do slide 7.

Os blocks chegam aos SMs.

Cada block fica inteiro em um SM; vários podem residir juntos se houver recursos.



Os blocks da grid chegam aos SMs; cada block se divide em 8 warps.

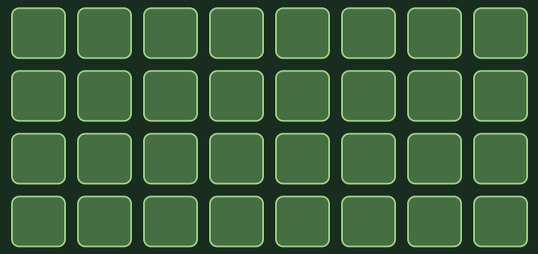
Esquema de distribuição, sem escala: o número de SMs e a atribuição real variam.

Warp: 32 threads avançam juntas.

Um block de 256 threads forma 8 warps.

BLOCK 0 · 256 THREADS → 8 WARPS

UMA WARP · 32 THREADS



Mesma instrução · dados diferentes

UM BLOCO DE PROCESSAMENTO DO SM

ESCALONADOR	escolhe uma warp pronta
warp A	espera memória
warp B	pronta → executa
PRÓXIMA INSTRUÇÃO DA WARP B	`C[i] = A[i] + B[i]`

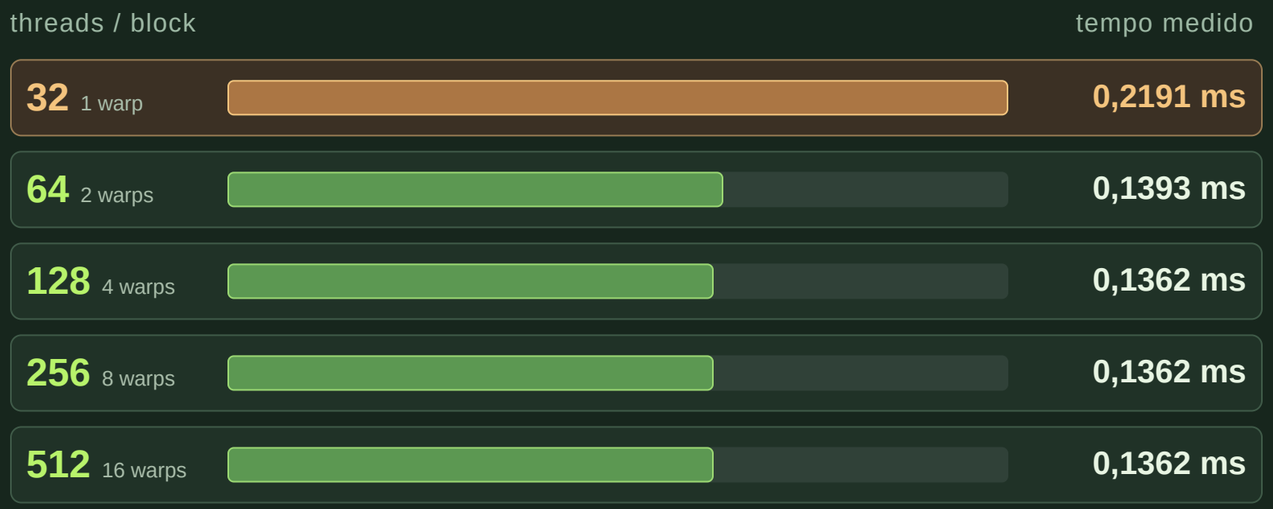
Se uma warp espera memória, outra pronta pode avançar no mesmo bloco de processamento.

Esquema conceitual de um bloco de processamento do SM.

O tamanho do block muda o tempo.

Mesmo kernel, 2 milhões de somas. Só mudamos as threads por block.

SÓ O KERNEL · TEMPO EM ms



HIPÓTESE PARA 32 THREADS

1 warp por block

São 62.500 blocks. O limite de blocks residentes pode deixar poucos warps no SM para esconder esperas.

DE 64 A 512 THREADS

Quase um platô

Os tempos variam cerca de 2%. Aumentar o block não garante mais trabalho útil por segundo.

O melhor tamanho depende do kernel e da GPU. 256 é um exemplo, não uma regra.

O dado precisa chegar à conta.

01 **Registradores** — por thread · on-chip

02 **Shared memory** — por block · on-chip

03 **Caches** — on-chip

04 **Memória global / VRAM** — mais distante

perto da execução — mais tráfego / custo

∑ **Compute-bound**

O limite é calcular.

↔ **Memory-bound**

O limite é mover dados.

Em GPU, mover dados pode custar mais do que fazer a conta.

> Uma thread não acessa diretamente os **registradores privados** de outra thread.

Três kernels ou um?

$$y_i = (x_i + \alpha)\beta + \gamma$$

3 KERNELS SEPARADOS

Os resultados **a** e **b** viram vetores intermediários na VRAM.

01 · add_kernel	`a[i] = x[i] + alpha;`
02 · multiply_kernel	`b[i] = a[i] * beta;`
03 · final_add_kernel	`y[i] = b[i] + gamma;`

TEMPO NA GPU · 3 CHAMADAS — **0,2774 ms**

1 KERNEL FUNDIDO

As três etapas usam um valor local **v** antes de gravar **y**.

fused_kernel

```
float v = x[i];
v = v + alpha;
v = v * beta;
v = v + gamma;
y[i] = v;
```

TEMPO NA GPU · 1 CHAMADA — **0,0943 ms**

Mesmo resultado numérico; o kernel fundido foi 2,94× mais rápido.

Fusão: uma passagem.

MESMA CONTA: $y_i = (x_i + \alpha)\beta + \gamma$

TIPO USADO NO EXEMPLO
float32 = 4 bytes por valor

SEM FUSÃO

Cada etapa grava um vetor; a próxima o lê.

$a_i = x_i + \alpha$	lê x[i] · grava a[i] na VRAM
$b_i = a_i\beta$	lê a[i] · grava b[i] na VRAM
$y_i = b_i + \gamma$	lê b[i] · grava y[i] na VRAM

24 bytes/elemento · 6 acessos × 4 bytes

COM FUSÃO

As três contas acontecem antes de gravar a saída.

VALOR LOCAL NO REGISTRADOR

$$v = (x_i + \alpha)\beta + \gamma$$

$$y_i = v$$

lê x[i] · grava y[i] na VRAM

8 bytes/elemento · 2 acessos × 4 bytes

24 bytes → 8 bytes
tráfego lógico por elemento*

3 → 1
kernel launches

3 → 3
FLOPs matemáticos por elemento

* Acessos lógicos; caches podem alterar o tráfego físico. Fusão excessiva pode pressionar registradores e reduzir occupancy.

Mesmos FLOPs. Menos tempo.

3 KERNELS SEPARADOS

0,2774 ms → 1 KERNEL FUNDIDO **0,0943 ms**

≈ 2,94×
mais rápido

Medição desta execução · resultados variam conforme a GPU

Como ficou ~3× mais rápido sem reduzir nenhum FLOP?

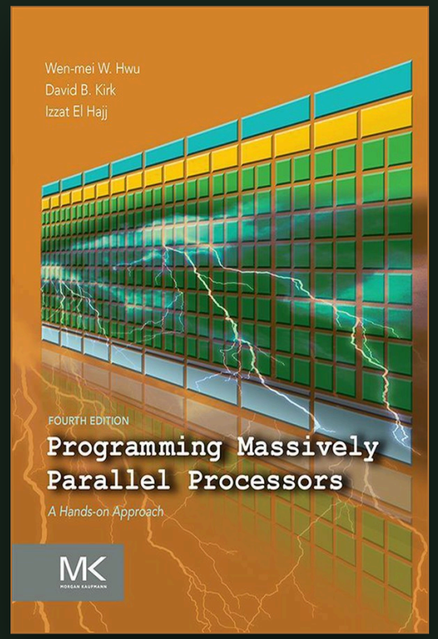
- menos tráfego de VRAM
- intermediários no registrador
- menos launches

O que muda então?

O **caminho dos dados**.

Obrigado!

Para aprofundar: programação paralela e arquitetura de GPUs.

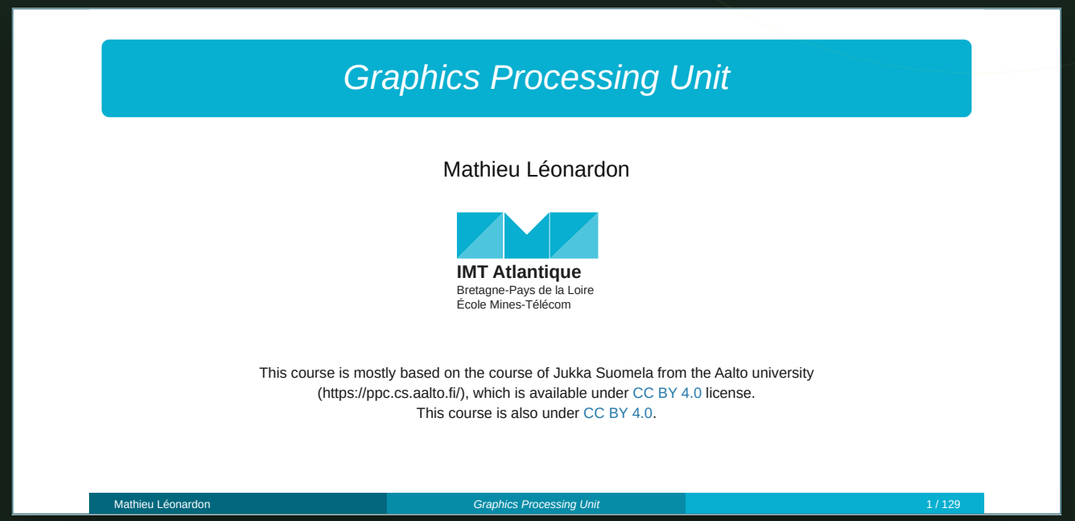


LIVRO

Programming Massively Parallel Processors

Wen-mei W. Hwu · David B. Kirk · Izzat El Hajj

4ª edição · Morgan Kaufmann



SLIDES

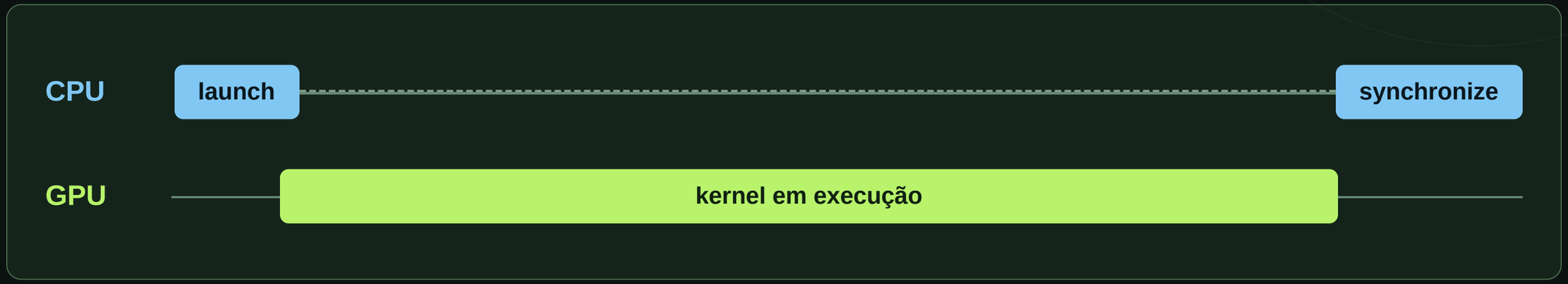
Graphics Processing Unit

Mathieu Léonardon · IMT Atlantique

Material disponibilizado sob CC BY 4.0

Lançar ≠ terminar.

O host normalmente enfileira o kernel e segue; a GPU executa depois.



01

vector_add

uma thread por elemento

02

só kernel

mede a execução na GPU

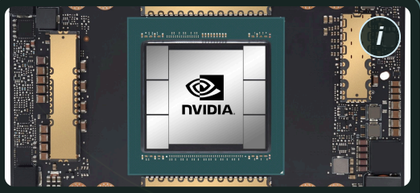
03

fluxo completo

cópias e lançamento também custam tempo

Se uma warp espera, outra avança.

No block 0, as 256 threads formam 8 warps de 32 threads.



NVIDIA A100 · Ampere

BLOCK 0 · 8 WARPS

warp 0 0–31 · espera memória	**warp 1** 32–63 · pronta
warp 2 64–95	**warp 3** 96–127
warp 4 128–159	**warp 5** 160–191
warp 6 192–223	**warp 7** 224–255

Escalonador de warps → escolhe a warp 1, pronta para executar

→

UM SM DA A100

64 núcleos CUDA = **4 blocos de processamento × 16**

bloco de processamento 1 **16 núcleos CUDA**	bloco de processamento 2 **16 núcleos CUDA**	bloco de processamento 3 **16 núcleos CUDA**	bloco de processamento 4 **16 núcleos CUDA**

warp 1 → **bloco de processamento 1** (subpartição do SM)

mesma instrução, dados diferentes: `C[i] = A[i] + B[i]`

1º ciclo threads 32–47

2º ciclo threads 48–63

A warp 0 espera memória; o escalonador põe a warp 1 para avançar nas unidades do SM.

Se um block tiver 48 threads: 2 warps; na última, 16 posições ficam inativas.